

1. Definition

React.memo is a higher order component (HOC) used to optimize performance in React applications.

It prevents unnecessary re-rendering of functional components when the props remain unchanged.

2. Need of Memo

- Improves application performance.
 - Avoids unnecessary component rendering.
 - Useful in large applications with many components.
 - Reduces CPU usage and rendering time.
-

3. Working of React.memo

- React compares the previous props and new props.
 - If props are the same, React skips re-rendering the component.
 - If props change, the component re-renders normally.
-

4. Syntax

```
const MemoComponent = React.memo(Component);
```

5. Example

```
import React from "react";
```

```
const Student = React.memo(function Student(props) {  
  console.log("Rendering Student Component");  
  return <h2>Student Name: {props.name}</h2>;  
});
```

```
export default Student;
```

Explanation

- If the parent component updates but the name prop remains the same, the Student component will not re-render.
-

6. Real-life Example

In a shopping website, the product list component may contain many items.

Using **React.memo** prevents re-rendering of product components when only unrelated parts of the page update.

7. Advantages

- Improves performance of React applications.
- Reduces unnecessary DOM updates.
- Helps in efficient component rendering.

Introduction to Refs: Refs, Refs with Class Components, Forwarding Refs and Portals 1

1. Refs (References)

- **Refs are used in React to access and interact with DOM elements directly.**
- **They allow developers to focus, select, or modify DOM elements without using state or props.**
- **Refs are created using `React.createRef()`.**

Example

```
class Example extends React.Component {
  constructor() {
    super();
    this.inputRef = React.createRef(); }
  focusInput = () => {
    this.inputRef.current.focus(); }
  render() {
    return (
      <div>
        <input ref={this.inputRef} />
        <button onClick={this.focusInput}>Focus</button>
      </div> ); } }
```

Example use:

Automatically focusing the input field in a login form.

2. Refs with Class Components

- **In class components, refs are created in the constructor using `React.createRef()`.**
- **The ref is attached to a DOM element using the `ref` attribute.**
- **The DOM element can be accessed using `ref.current`.**

Steps

- 1. Create ref using `React.createRef()`.**
- 2. Attach ref to an element.**
- 3. Access DOM element using `ref.current`.**

Example

```
class MyComponent extends React.Component {
  constructor() {
    super();
    this.myRef = React.createRef(); }
  componentDidMount() {
    this.myRef.current.style.color = "red"; }
  render() {
    return <h1 ref={this.myRef}>Hello React</h1>; } }
```

3. Forwarding Refs

- **Forwarding Refs allows a parent component to pass a ref to a child component.**
- **It is used when the parent needs direct access to the DOM element inside the child component.**
- **Implemented using `React.forwardRef()`.**

Introduction to Refs: Refs, Refs with Class Components, Forwarding Refs and Portals2

Example

```
const Input = React.forwardRef((props, ref) => {  
  return <input ref={ref} />;  
});
```

Example use:

A parent component can focus the input field inside a child component.

4. Portals

- **Portals allow React components to render children outside the main DOM hierarchy.**
- **Used for modals, dialogs, tooltips, and pop-up messages.**
- **Created using ReactDOM.createPortal().**

Example

```
ReactDOM.createPortal(  
  <h1>Popup Message</h1>,  
  document.getElementById("modal-root") );
```

Example use:

Displaying a modal dialog box outside the main application container.

1. Definition

A Higher Order Component (HOC) is an advanced technique in React used to reuse component logic. It is a function that takes a component as input and returns a new enhanced component.

2. Purpose of HOC

- **To reuse component logic.**
 - **To avoid code duplication.**
 - **To add extra functionality to components.**
 - **To separate logic from UI components.**
-

3. Syntax of HOC

const EnhancedComponent = higherOrderComponent(WrappedComponent);

- **higherOrderComponent** → **function that adds extra features.**
 - **WrappedComponent** → **original component.**
 - **EnhancedComponent** → **new component with additional functionality.**
-

4. Example of Higher Order Component

```
import React from "react";
function withCounter(WrappedComponent) {
  class NewComponent extends React.Component {
    constructor(props) {
      super(props);
      this.state = { count: 0 };
    }
    increment = () => {
      this.setState({ count: this.state.count + 1 });
    }
    render() {
      return (
        <WrappedComponent
          count={this.state.count}
          increment={this.increment} />
      );
    }
  }
  return NewComponent;
}
```

5. Using the HOC

```
function ClickCounter(props) {
  return (
    <button onClick={props.increment}>
      Clicked {props.count} times
    </button>
  );
}
export default withCounter(ClickCounter);
```

Explanation

- **withCounter is the Higher Order Component.**
- **It provides count state and increment function.**
- **ClickCounter receives these as props.**

7. Advantages

- **Improves code reuse.**
- **Makes code modular and maintainable.**
- **Separates logic from UI.**

1. Rendering Props

- **Render Props is a technique in React used to share code between components using a prop whose value is a function.**
- **Instead of passing static data, a function is passed as a prop, and the component calls this function to render UI.**
- **It helps in code reuse and flexible component design.**

Example

```
class Counter extends React.Component {
  state = { count: 0 };
  increment = () => {
    this.setState({ count: this.state.count + 1 });
  };
  render() {
    return this.props.render(this.state.count, this.increment);
  }
}
```

Using the Component

```
<Counter render={(count, increment) => (
  <button onClick={increment}>Clicked {count} times</button> )} />
```

Example use:

Sharing counter logic between multiple components.

2. Context in React

- **Context is used to share data globally between components without passing props manually at every level.**
- **It helps to avoid prop drilling (passing props through many components).**
- **Context is created using `React.createContext()`.**

Example

```
const UserContext = React.createContext();
function App() {
  return (
    <UserContext.Provider value="Nidhi">
      <Child />
    </UserContext.Provider> );
}
function Child() {
  return (
    <UserContext.Consumer>
      {value => <h1>Hello {value}</h1>}
    </UserContext.Consumer> );
}
```

Example use:

Sharing user login information, theme settings, or language preferences across multiple components.

HTTP: HTTP and React, GET and React, POST and React.

1. HTTP and React

- **HTTP (HyperText Transfer Protocol) is a protocol used for communication between client and server.**
- **In React applications, HTTP is used to send requests to APIs and receive data from servers.**
- **React uses `fetch()` or libraries like `Axios` to perform HTTP requests.**
- **These requests help React applications retrieve, send, update, or delete data from databases.**

Example

```
fetch("https://api.example.com/users")
  .then(response => response.json())
  .then(data => console.log(data));
```

Example use:

A React application fetching student data from a server and displaying it on the webpage.

2. GET and React

- **The GET method is used to retrieve data from a server.**
- **It does not change server data and is mainly used to read information.**
- **In React, GET requests are commonly used to fetch API data and display it in components.**

Example

```
import React, { useEffect, useState } from "react";
function Users() {
  const [users, setUsers] = useState([]);
  useEffect(() => {
    fetch("https://api.example.com/users")
      .then(res => res.json())
      .then(data => setUsers(data)); }, []);
  return (
    <div>
      {users.map(user => (
        <p key={user.id}>{user.name}</p>      ))}
    </div> ); }

```

Example use:

Displaying a list of products or students retrieved from a server.

3. POST and React

- **The POST method is used to send data from the client to the server.**
- **It is commonly used to store or submit data such as form information.**
- **React sends POST requests using `fetch()` or `Axios` with request body data.**

Example

```
fetch("https://api.example.com/users", {
  method: "POST",
  headers: {
    "Content-Type": "application/json" },
  body: JSON.stringify({
    name: "Nidhi",
    email: "nidhi@email.com" }) });
```

Example use: Submitting a student registration form where user details are sent to the server and stored in a database.

